

Conectando Airflow a Bancos de Dados

Providers, Connections e SQLExecuteQueryOperator

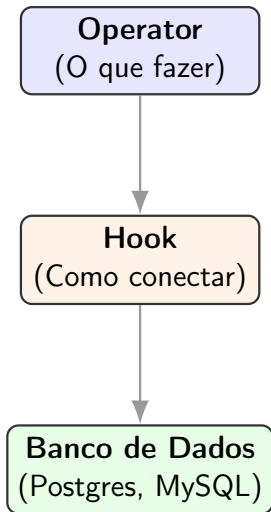
Disciplina de Extração de Dados

IBMEC

12 de maio de 2026

- Aprender como o Apache Airflow se comunica de forma segura e nativa com sistemas externos (Bancos de Dados).
- Compreender o conceito de **Providers**, **Hooks** e **Connections**.
- Analisar e executar dois *pipelines* práticos de Engenharia de Dados:
 - **Cenário 1:** Banco local leve (**SQLite**) para desenvolvimento rápido.
 - **Cenário 2:** Banco relacional robusto (**PostgreSQL**) padrão da indústria.

Arquitetura de Integração (Providers e Hooks)



- O Airflow **não** vem com todos os drivers do mundo instalados por padrão.
- **Providers**: Pacotes extras que instalamos (ex: `apache-airflow-providers-postgres`).
- **Hooks**: A "mágica" interna. É uma interface de baixo nível que gerencia a conexão física, credenciais e portas com o sistema externo.
- **Operators**: A abstração de alto nível que o engenheiro usa no código para instruir o **Hook** a executar algo.

Admin → Connections

Conn Id: postgres_default

Conn Type: Postgres

Host: localhost

Login: airflow

Password: *****

- No código da DAG, **nunca** escrevemos senhas em texto puro (*hardcoded*).
- Passamos apenas um `conn_id` (ID da Conexão).
- O Airflow guarda as credenciais criptografadas em seu próprio banco.
- **Na prática:** Abra a interface Web, vá no menu superior **Admin** → **Connections** e configure o banco de dados antes de rodar o código.

Detalhando os Campos da Conexão (Formulário UI)

Conn Id: postgres_default	<i>Nome usado no Python</i>
Conn Type: Postgres	<i>O motor do Banco</i>
Host: host.docker.internal	<i>Endereço / IP</i>
Schema: postgres	<i>Nome do Database</i>
Login: airflow	<i>Usuário</i>
Port: 5432	<i>Porta de Acesso</i>

- Ao clicar no botão azul de **Add Connection**, os campos cruciais são:
- **Conn Id**: O identificador exato que usamos no código (postgres_default). É como o Airflow acha a credencial correta.
- **Conn Type**: O tipo do banco (Postgres, MySQL). **Atenção**: se a opção não aparecer, falta instalar o *Provider* respectivo no ambiente!
- **Host**: Onde o banco está (localhost ou host.docker.internal).
- **Schema, Login e Port**: Nome do banco, usuário e porta (**5432**).

O Padrão Moderno: SQLExecuteQueryOperator

- Nas versões mais antigas do Airflow, usávamos o PostgresOperator ou SqliteOperator.
- Agora, a forma moderna e unificada é o SQLExecuteQueryOperator.
- Ele serve para **qualquer banco de dados**. O que muda é apenas o conn_id!

```
1 from airflow.providers.common.sql.operators.sql import SQLExecuteQueryOperator
2
3 tarefa_sql = SQLExecuteQueryOperator(
4     task_id='inserir_vendas',
5     conn_id='postgres_default', # Define ONDE o SQL vai rodar
6     sql="INSERT INTO vendas (valor) VALUES (100.50);"
7 )
```

Cenário 1: SQLite (O Banco de Bolso)

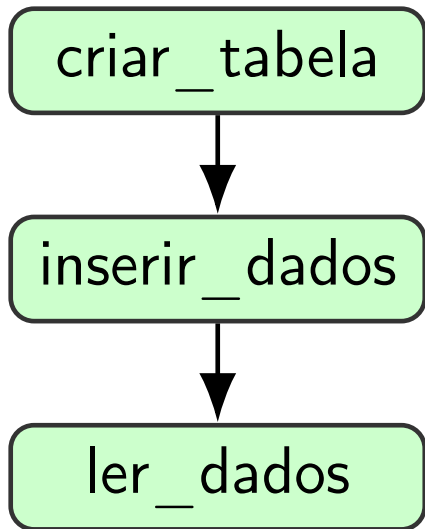
- **Vantagem:** O SQLite já vem nativo no Airflow. A conexão `sqlite_default` já existe e aponta para um arquivo local na sua máquina.

```
1  criar_tabela = SQLExecuteQueryOperator(  
2      task_id='criar_tabela_usuarios',  
3      conn_id='sqlite_default',  
4      sql="""  
5          CREATE TABLE IF NOT EXISTS usuarios (  
6              id INTEGER PRIMARY KEY AUTOINCREMENT,  
7              nome TEXT NOT NULL,  
8              data_cadastro TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
9          );  
10     """)  
11 )
```

Cenário 2: PostgreSQL (Ambiente Real)

- **Vantagem:** Simula um servidor de Data Warehouse real. Se você usa Docker, ele já está rodando. Precisamos apenas apontar o `conn_id` para ele.

```
1  inserir_dados = SQLExecuteQueryOperator(  
2      task_id='inserir_dados_vendas',  
3      conn_id='postgres_default',  
4      sql="""  
5          INSERT INTO vendas (produto, valor) VALUES  
6          ('Notebook Airflow', 4500.00),  
7          ('Monitor Docker', 1200.50);  
8      """  
9  )
```

- Ambos os scripts possuem o fluxo: `criar_tabela` » `inserir_dados` » `ler_dados`.
- A tarefa de leitura utiliza o parâmetro `show_return_value_in_logs=True`.
- **Ação do Aluno:**
 - 1 Crie e ative a DAG.
 - 2 Aguarde as 3 tarefas ficarem verdes.
 - 3 Clique em `ler_dados` → Logs.
 - 4 Visualize as linhas inseridas no banco impressas no terminal da interface gráfica!

Próximos Passos: Caminho para o Projeto Integrador

- Agora que dominamos a **Orquestração** (Airflow) e o **Armazenamento** (Bancos de Dados), o objetivo final da disciplina é construir um Pipeline de Engenharia de Dados real (ETL).
- **1. Comunicação entre Tasks (XComs)**
 - Como quebrar o isolamento das **tasks**: ensinando uma tarefa que extraiu um dado a passar esse resultado para a próxima tarefa na fila.
- **2. Extração Real via API (O "E" do ETL)**
 - Deixar de inserir dados manuais e começar a usar o Python (biblioteca requests) para consumir JSONs dinâmicos da internet (ex: finanças, clima).
- **3. Projeto Integrador (O Pipeline Completo)**
 - Unir tudo em uma DAG autônoma: Extrair dados da API → Limpar os dados brutos usando Pandas → Salvar o dado tratado no nosso PostgreSQL/SQLite.